



SJSU CMPE 172

CMPE 172-02 – MILESTONE 2 REPORT

Online Appointment Scheduling System: Interview Scheduler

Name: Charlie Banh

Student ID: 015345473

Date: October 8, 2026

Chosen scenario: Interview Scheduler to help with resume reviews and mock interviews for software engineers

GitHub repository (milestone tag): <https://github.com/c-banh-mi/cmpe172-interview-scheduler/tree/milestone-2>

Code-walkthrough video: <https://drive.google.com/file/d/10eYO288ZeQLtlxiJdYc3K5R9li3TWqV-/view?usp=sharing>

1. Overview

Milestone 2 adds login with server-side sessions and role-based access control on top of Milestone 1 along with the full set of customer and provider booking features. It also implements concurrency control, so two users booking the same slot at the same time won't cause double-booking.

2. Design and Implementation

2.1 Web interface

Bootstrap 5 + plain JavaScript single-page app (static/index.html + app.js) calling the JSON API with fetch(); no build step

Page/View	Role	Endpoint(s) Called	Screenshot
Home	Anyone	GET /api/home	Figure 1
Login	Anyone	POST /api/auth/login, GET /api/auth/me	Figure 2
Available slots (filter + pagination)	Anyone	GET /api/slots?providerId&serviceId&date&page&size	Figure 3
Book appointment (form)	Customer	GET /api/slots/{id}, POST /api/customer/appointments	Figure 4
Confirmation	Customer	(201 response of the booking call; after a reload: GET /api/customer/appointments?scope=upcoming)	Figure 5
My appointments (upcoming/history, cancel)	Customer	GET /api/customer/appointments?scope=..., POST .../{id}/cancel	Figure 6
Provider: my slots (create/remove)	Provider	GET /api/services, GET / POST /api/provider/slots, DELETE /api/provider/slots/{id}	Figure 7
Provider: booked appointments	Provider	GET /api/provider/appointments?scope=...	Figure 8

A vanilla JavaScript single-page app was used because it avoids needing a heavy frontend build setup while letting the UI talk directly to the same JSON API that the integration tests use. This keeps the Front Controller pattern from Milestone 1. Screenshots of every page are in Appendix C, Figures 1-8 (taken from the running Docker app).

2.2 Frontend-to-backend request flow

When a user clicks confirm, the browser sends a POST request with the booking details, session cookie, and CSRF token. Spring Security checks the session and role first, then passes it to the controller and service layers. The service handles the database transaction and version check via JDBC, returning the created appointment details as JSON, so the frontend can display the confirmation view. Step by step:

1. **Spring Security Filter Chain:** intercepts the request first, validating the CSRF token, resolving the session to an authenticated AppUser principal, and enforcing the ROLE_CUSTOMER authorization rule
 2. **DispatcherServlet & Controller:** routes the request to CustomerController.book(), which validates incoming payloads using Jakarta Bean Validation (@Valid BookingRequest)
 3. **Service Layer:** delegates execution to BookingService (managing the retry loop) and SlotBooker (annotated with @Transactional(isolation = Isolation.READ_COMMITTED))
 4. **Repository & Database:** executes handwritten SQL queries via JdbcClient against PostgreSQL to atomically check versions, insert appointment records, and enforce constraints
 5. **Response:** service maps the newly created appointment into an AppointmentDto, returning HTTP 201 Created with JSON back to app.js, which renders the confirmation view
- Sequence diagram of this request with the retry and 409 paths: Appendix D, Figure 11 (docs/booking-sequence.png)
 - Block diagram with the Spring Security layer: Appendix D, Figure 10

2.3 Booking flow

Step	Code/SQL	If it fails
1. Read the slot	SELECT ... status, version ... FROM availability_slots WHERE id = :id	missing → 404; wrong service → 400; not OPEN or already started → 409
2. Rules	customer has no overlapping BOOKED appointment (SlotBooker.java:58)	overlap → 409
3. Claim the slot	UPDATE availability_slots SET status = 'BOOKED', version = version + 1 WHERE id = :id AND version = :version AND status = 'OPEN' (SlotRepository.java:102-119)	0 rows → race lost → BookingService retries
4. Create the appointment	INSERT INTO appointments (slot_id, service_id, customer_id, notes) VALUES (...) RETURNING id; status defaults to 'BOOKED' (AppointmentRepository.insert)	uq_appt_active_slot violation → DuplicateKeyException → 409 (BookingService.java:62-64)
5. Commit	COMMIT when SlotBooker.bookOnce returns (SlotBooker.java:44-69)	any exception → whole booking rolls back

- **Cancellation:** Initiated by an authenticated customer owning the appointment (AppointmentService.java:60-62). The service verifies ownership (non-owners receive 403), checks that the appointment status is BOOKED and has not started, performs a conditional cancellation update (appointments.cancel() == 0), and reopens the corresponding slot in a single atomic transaction.
- **Provider Slot Removal:** Providers can remove their upcoming slots as long as they are still OPEN. Attempting to remove a slot that has already been BOOKED fails with HTTP 409 Conflict. Removal is implemented as a soft delete (REMOVED), freeing up that start time for future scheduling.
- **COMPLETED Status:** A scheduled background job runs once a minute to automatically transition past BOOKED appointments to COMPLETED status, and the list queries already show an ended BOOKED appointment as COMPLETED, so the customer and provider dashboards are correct between runs.

2.4 Concurrency and transactions

Race condition

For example, customers Sam and Jordan simultaneously click “Book” on the exact same open slot. Without concurrency protection, both threads would read the slot as OPEN and successfully insert overlapping active appointments.

ACID in the booking use case

Property	How it applies to booking
Atomicity	All-or-nothing execution; if slot claiming, overlap checks, or appointment insertion fails, the entire transaction rolls back
Consistency	Database constraints (such as foreign keys and unique indexes) and business rules ensure invalid states (like double-booking or booking past slots) never persist
Isolation	READ COMMITTED isolation prevents dirty reads by ensuring transactions only see committed data. However, isolation alone doesn’t prevent double-booking; that requires version checking combined with PostgreSQL row locking during the UPDATE statement.
Durability	Once a booking transaction commits, PostgreSQL guarantees that the appointment and slot state updates are permanently saved to disk



Isolation level

READ COMMITTED (PostgreSQL default) is explicitly configured via `@Transactional(isolation = Isolation.READ_COMMITTED)` (SlotBooker.java:36-69). With version checking, the loser of a race updates 0 rows and throws an optimistic locking exception; stricter levels like REPEATABLE READ would raise serialization failures instead without adding extra safety, while SERIALIZABLE would introduce excessive transaction aborts and retries.

Concurrency control: optimistic version check

If two users try to book the same slot simultaneously, PostgreSQL handles it by letting one update proceed and forcing the second one to wait until the first commits. Once the first finishes, the second runs its update query, but because the version number has already been incremented, the WHERE clause no longer matches and 0 rows are updated. This lets our code detect the conflict immediately without needing upfront pessimistic locks like SELECT FOR UPDATE.

src/main/java/edu/sjsu/cmpe172/scheduler/repository/SlotRepository.java:110-119

```
110     public int markBooked(long id, int expectedVersion) {
111         return jdbc.sql("""
112             UPDATE availability_slots
113             SET status = 'BOOKED', version = version + 1
114             WHERE id = :id AND version = :version AND status = 'OPEN'
115             """)
116             .param("id", id)
117             .param("version", expectedVersion)
118             .update();
119     }
```

Database backstop

The partial unique index `uq_appt_active_slot` (UNIQUE (slot_id) WHERE status = 'BOOKED', schema.sql:72-74) acts as a safety backstop kept from Milestone 1. Even if application-level checks or version logic were somehow bypassed, the database constraint rejects duplicate active bookings with a `DuplicateKeyException` (BookingService.java:62-64), which is centrally mapped to HTTP 409.

Retry

BookingService wraps booking attempts in a retry loop (`MAX_ATTEMPTS = 3`, BookingService.java:39-77). When a thread loses the race, the retry loop catches the optimistic locking failure and checks the slot again. If another user successfully grabbed it, the retry sees it as BOOKED and immediately fails with a 409 conflict. The retry loop is really only useful if the slot's version changed due to some minor non-booking reason while the slot remained open. Unique-index violations fail immediately with HTTP 409 without retrying.

src/main/java/edu/sjsu/cmpe172/scheduler/service/BookingService.java:46-66

```
46     for (int attempt = 1; ; attempt++) {
47         try {
48             long id = booker.bookOnce(customer.id(), req);
49             log.info("Customer {} booked slot {} (appointment {}, attempt {})",
50                 customer.username(), req.slotId(), id, attempt);
51             return appointments.findViewById(id)
52                 .map(DtoMapper::toDto)
53                 .orElseThrow(() -> new NotFoundException("Appointment " + id + " not found"));
54         } catch (OptimisticLockingFailureException e) {
55             // Lost a race. Retrying re-reads the slot: if the winner booked it, the next
56             // attempt fails fast with 409; if the change was harmless, we can still book.
57             log.info("Version conflict booking slot {} (attempt {}/{})", req.slotId(), attempt, MAX_ATTEMPTS);
58             if (attempt == MAX_ATTEMPTS) {
59                 throw new SlotUnavailableException("Slot " + req.slotId() + " is busy, please try again");
60             }
61             backOff(attempt);
62         } catch (DuplicateKeyException e) {
63             // The unique index caught a second active booking for this slot.
64             throw new SlotUnavailableException("Slot " + req.slotId() + " is no longer available");
65         }
66     }
```

2.5 Role-based access (RBAC)

When a user logs in, Spring Security verifies their credentials against the database using BCrypt and loads their role (ROLE_CUSTOMER or ROLE_PROVIDER, read from the users.role column) and provider ID via the user repository. This creates an active session cookie (JSESSIONID), and protected requests require this cookie; POST and DELETE requests also need the X-XSRF-TOKEN header.

URL security rules in SecurityConfig.java:56-66 and the JSON 401/403 handlers in SecurityConfig.java:80-84 are shown:

src/main/java/edu/sjsu/cmpe172/scheduler/repository/UserRepository.java:29-39

```
29 public Optional<AppUser> findByUsername(String username) {
30     return jdbc.sql("""
31         SELECT u.id, u.username, u.password_hash, u.full_name, u.role, p.id AS provider_id
32         FROM users u
33         LEFT JOIN providers p ON p.user_id = u.id
34         WHERE u.username = :username
35         """)
36         .param("username", username)
37         .query(MAPPER)
38         .optional();
39 }
```

src/main/java/edu/sjsu/cmpe172/scheduler/security/SecurityConfig.java:56-66

```
56 .authorizeHttpRequests(auth -> auth
57     .requestMatchers("/api/customer/**").hasRole("CUSTOMER")
58     .requestMatchers("/api/provider/**").hasRole("PROVIDER")
59     .requestMatchers(HttpMethod.GET, "/api/home", "/api/slots", "/api/slots/*", "/api/services",
"/api/providers")
60     .permitAll()
61     .requestMatchers("/api/auth/**").permitAll()
62     .requestMatchers("/actuator/health", "/actuator/info").permitAll()
63     .requestMatchers(HttpMethod.GET, "/", "/index.html", "/app.js", "/app.css", "/favicon.ico")
64     .permitAll()
65     .requestMatchers("/error").permitAll()
66     .anyRequest().authenticated())
```

src/main/java/edu/sjsu/cmpe172/scheduler/security/SecurityConfig.java:80-84

```
80 .exceptionHandling(ex -> ex
81     .authenticationEntryPoint((req, res, e) ->
82         writeError(req, res, HttpStatus.UNAUTHORIZED, "Please log in"))
83     .accessDeniedHandler((req, res, e) ->
84         writeError(req, res, HttpStatus.FORBIDDEN, "Your role is not allowed to do this")))
```

Endpoint	Method	Allowed Role	Not logged in	Wrong role
/api/home, /api/slots, /api/slots/{id}, /api/services, /api/providers	GET	Anyone	-	-
/actuator/health, /actuator/info	GET	Anyone	-	-
/api/auth/login	POST	Anyone	-	-
/api/auth/me	GET	Anyone (204 if not logged in)	-	-
/api/auth/logout	POST	Logged in	204 (no-op)	-
/api/customer/appointments	POST/GET	Customer	401	403
/api/customer/appointments/{id}/cancel	POST	Customer (owner)	401	403
/api/provider/slots	GET/POST	Provider	401	403
/api/provider/slots/{id}	DELETE	Provider (owner)	401	403
/api/provider/appointments	GET	Provider	401	403

Every POST/DELETE must also send the X-XSRF-TOKEN header. Without it the CSRF filter answers 403 before the role check; with it, an anonymous request gets 401 (both verified against the running app). Owner rules: AppointmentService.java:60-62 (cancel) and ProviderSlotService.java:82-84 (remove slot) → 403.

2.6 Validation and global exception handling

GlobalExceptionHandler.java:26-77; every error body is {timestamp, status, error, message, path}, no stack traces.

Situation	HTTP status
Bad input, failed @Valid validation, malformed JSON, page size too large	400
Not logged in	401
Wrong role; cancelling someone else's appointment; removing another provider's slot; missing CSRF token	403
Slot or appointment not found	404
Slot already booked, lost race after retries, overlap, removing a booked slot, unique-index violation	409
Anything unexpected	500 (generic)

3. Design Decisions

- **Optimistic Version Check vs. SELECT ... FOR UPDATE:** chosen because it avoids holding pessimistic row locks when the slot is read (the version check detects a conflicting update instead), maximizing concurrency
 - Cost: requires retry handling when race conditions occur
- **READ COMMITTED vs. Stricter Isolation Levels:** chosen to balance consistency and performance, preventing dirty reads while avoiding excessive serialization aborts
 - Cost: relies on application-level checks and version columns to manage concurrency safely
- **Spring Security vs. hand-rolled interceptor:** used Spring Security because it handles session management, CSRF protection, and role checks out of the box instead of writing custom auth filters
 - Cost: requires extra configuration to return proper JSON error codes instead of HTML redirects
- **Plain-JS SPA vs. Thymeleaf / React:** chosen to eliminate complex frontend build pipelines while enabling direct API testing against the exact same JSON REST endpoints
 - Cost: manual DOM manipulation in vanilla JavaScript for dynamic view rendering
- **Seeding Only Empty Tables:** chosen to preserve test and development data across container restarts without destructive wipes
 - Cost: requires conditional (WHERE NOT EXISTS) inserts in the seed script (seed.sql)

4. Challenges and Solutions

- **Docker/Testcontainers networking:** Testcontainers could not reach the Docker daemon when running tests.
 - Cause: Docker Desktop was not running
 - Fix: added a localdb test profile so local runs used a local PostgreSQL while CI ran Testcontainers; once Docker Desktop was running, mvn -B package also ran with Testcontainers locally (67 tests, 0 failures)
- **CSRF token and authentication errors:** Unauthenticated POST requests were returning 403 Forbidden instead of 401 Unauthorized.
 - Cause: the CSRF filter evaluated the missing token before authentication checks ran

- Resolution: confirmed that this is Spring Security's normal check order; the frontend always sends the header, so a logged-out user in the app gets 401 as expected, and the RBAC section documents the 403 case
- **Logged-out visitors saw a "Log out" button next to "Log in"**
 - Cause: Bootstrap's d-flex class forces the element to display, overriding the hidden attribute
 - Fix: one CSS rule in app.css that makes hidden always win, commit bd61e6c

5. Testing and Results

mvn -B package: 67 tests, 0 failures; CI green (mvn -B verify with Testcontainers)

Test	What it checks
ConcurrentBookingTest (11)	Two threads book one slot at once, repeated 10 times: exactly 1 success, 1 conflict (409), exactly 1 BOOKED row
BookingServiceTest (11)	Booking rules: missing/non-OPEN slot, overlap, retry behavior
AppointmentServiceTest (8)	Owner-only cancel → 403; status changes
ProviderSlotServiceTest (10), SlotServiceTest (2)	Slot create/remove rules, paging, and filter validation
AuthIntegrationTest (9)	Login/logout, BCrypt hashes, 401 without a session, 403 wrong role, CSRF token required
BookingFlowIntegrationTest (9)	Over HTTP: book → 201, taken slot → 409, invalid body → 400, unknown slot → 404, owner-only cancel (403), provider create/remove, COMPLETED status
ApiIntegrationTest (7)	Public browse, filter, LIMIT/OFFSET paging, single slot, 400 on bad paging

mvn -B package runs 67 tests across 8 test classes with 0 failures; the run shown below used the localdb profile; a later plain run with Testcontainers also passed, 67 tests, 0 failures); CI runs the same tests with Testcontainers and is green

mvn -B package (docs/test-output.txt)

```
Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.005 s SlotServiceTest
Tests run: 8, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.152 s AppointmentServiceTest
Tests run: 9, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 5.143 s BookingFlowIntegrationTest
Tests run: 9, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.153 s AuthIntegrationTest
Tests run: 11, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.339 s ConcurrentBookingTest
Tests run: 7, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.609 s ApiIntegrationTest
Tests run: 11, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.154 s BookingServiceTest
Tests run: 10, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.059 s ProviderSlotServiceTest
Tests run: 67, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
Total time: 12.502 s
```

ConcurrentBookingTest log (docs/test-output.txt)

```
Two-thread booking race (ConcurrentBookingTest, 10 repetitions), from the test log:
 6 Customer jordan.dev booked slot 1 (appointment 2, attempt 1)
 4 Customer sam.dev booked slot 1 (appointment 2, attempt 1)
10 Version conflict booking slot 1 (attempt 1/3)
(each repetition: one 'Version conflict' for the loser, which then got 409; the winner varies)
```

Platform Solutions Resources Open Source Enterprise Pricing

Sign in Sign up

c-banh-mi / cmpe172-interview-scheduler

<> Code Issues Pull requests **Actions** Projects Security and quality Insights

Actions

All workflows

CI

Management

Caches

All workflows

Showing runs from all workflows

Filter workflow runs

18 workflow runs

Workflow	Event	Status	Branch	Actor	Run Time
Hide the Log out button for logged-out visitors	main	Success	main	c-banh-mi	2 minutes ago, 39s
Correct Docker notes: WSL integration was on...	main	Success	main	c-banh-mi	12 minutes ago, 41s
Lay out the block diagram top to bottom so it...	main	Success	main	c-banh-mi	Today at 10:02 AM, 48s
Add Milestone 2 report docs: booking sequen...	main	Success	main	c-banh-mi	Today at 9:04 AM, 40s
Add Milestone 2 summary, test results and o...	main	Success	main	c-banh-mi	Today at 11:55 PM, 42s

Figure 9. GitHub Actions: CI green on every Milestone 2 commit (mvn -B verify with Testcontainers)

src/test/java/edu/sjsu/cmpe172/scheduler/ConcurrentBookingTest.java:53-85

```

53 // Both threads wait at the barrier, then call book() at the same moment.
54 CyclicBarrier startTogether = new CyclicBarrier(2);
55 List<Callable<AppointmentDto>> attempts = List.of(
56     () -> { startTogether.await(); return bookingService.book(sam, req); },
57     () -> { startTogether.await(); return bookingService.book(jordan, req); });
58
59 int succeeded = 0;
60 List<Throwable> failures = new ArrayList<>();
61 ExecutorService pool = Executors.newFixedThreadPool(2);
62 try {
63     for (Future<AppointmentDto> result : pool.invokeAll(attempts, 30, TimeUnit.SECONDS)) {
64         try {
65             result.get();
66             succeeded++;
67         } catch (ExecutionException e) {
68             failures.add(e.getCause());
69         }
70     }
71 } finally {
72     pool.shutdownNow();
73 }
74
75 assertThat(succeeded).as("successful bookings").isEqualTo(1);
76 assertThat(failures).hasSize(1);
77 assertThat(failures.get(0)).assertInstanceOf(SlotUnavailableException.class); // -> HTTP 409
78
79 // The database agrees: one active appointment, slot BOOKED, version bumped exactly once.
80 long active = jdbc.sql("SELECT COUNT(*) FROM appointments WHERE slot_id = :id AND status = 'BOOKED'")
81     .param("id", slotId).query(Long.class).single();
82 Slot after = slots.findById(slotId).orElseThrow();
83 assertThat(active).isEqualTo(1);
84 assertThat(after.status()).isEqualTo(Slot.BOOKED);
85 assertThat(after.version()).isEqualTo(versionBefore + 1);

```


6. AI Tools Used

Claude Code wrote the Milestone 2 codebase largely autonomously based on the milestone requirements and architecture spec. I reviewed the implementation, checked DECISIONS.md, and tested the app and test suite to ensure everything is functional.

7. Additional Information

Demo logins (password password123): customers sam.dev, jordan.dev; providers alice.mentor, raj.mentor, maria.mentor

Run: `cp .env.example .env`, then `docker compose up --build` (see README)



Appendix

A. Schema changes since Milestone 1

- DROP TABLE lines removed and tables/indexes created IF NOT EXISTS, so data survives restarts
- uq_slot_provider_start changed from a UNIQUE constraint to a partial unique index WHERE status <> 'REMOVED', so a removed slot frees that start time
- seed.sql inserts only into empty tables
- Columns, CHECKs, FKs and uq_appt_active_slot are unchanged

B. Key code snippets

All snippets with file:line: docs/m2-report-material.md

src/main/java/edu/sjsu/cmpe172/scheduler/service/SlotBooker.java:44-69

```

44     @Transactional(isolation = Isolation.READ_COMMITTED)
45     public long bookOnce(long customerId, BookingRequest req) {
46         Slot slot = slots.findById(req.slotId())
47             .orElseThrow(() -> new NotFoundException("Slot " + req.slotId() + " not found"));
48
49         if (slot.serviceId() != req.serviceId()) {
50             throw new IllegalArgumentException("Slot " + slot.id() + " is not offered for service " + req.serviceId());
51         }
52         if (!Slot.OPEN.equals(slot.status())) {
53             throw new SlotUnavailableException("Slot " + slot.id() + " is no longer available");
54         }
55         if (!slot.startTime().isAfter(LocalDate.now(clock))) {
56             throw new SlotUnavailableException("Slot " + slot.id() + " has already started");
57         }
58         if (appointments.customerHasOverlap(customerId, slot.startTime(), slot.endTime())) {
59             throw new ConflictException("You already have an appointment at that time");
60         }
61
62         // The concurrency check: succeeds only if the row still has the version we read above.
63         if (slots.markBooked(slot.id(), slot.version()) == 0) {
64             throw new OptimisticLockingFailureException("Slot " + slot.id() + " changed while booking");
65         }
66         // Backstop: the partial unique index uq_appt_active_slot rejects a second BOOKED row
67         // for this slot even if the check above were ever bypassed (DuplicateKeyException).
68         return appointments.insert(slot.id(), slot.serviceId(), customerId, req.notes());
69     }

```

src/main/java/edu/sjsu/cmpe172/scheduler/service/AppointmentService.java:55-73

```

55     @Transactional(isolation = Isolation.READ_COMMITTED)
56     public AppointmentDto cancel(AppUser customer, long appointmentId) {
57         Appointment appt = appointments.findById(appointmentId)
58             .orElseThrow(() -> new NotFoundException("Appointment " + appointmentId + " not found"));
59
60         if (appt.customerId() != customer.id()) {
61             throw new ForbiddenException("You can only cancel your own appointments");
62         }
63         if (!Appointment.BOOKED.equals(appt.status())) {
64             throw new ConflictException("Appointment " + appointmentId + " is already " + appt.status());
65         }
66         if (!appt.startTime().isAfter(LocalDate.now(clock))) {
67             throw new ConflictException("Appointments that have already started cannot be cancelled");
68         }
69         // Conditional update: if two cancel requests race, only one changes the row.
70         if (appointments.cancel(appointmentId) == 0) {
71             throw new ConflictException("Appointment " + appointmentId + " was already cancelled");
72         }
73         slots.reopen(appt.slotId());

```

src/main/resources/schema.sql:72-74

```

72 -- DOUBLE-BOOKING GUARD: at most one *active* (BOOKED) appointment per slot.
73 -- Cancelled rows stay for history and don't block rebooking the slot.
74 CREATE UNIQUE INDEX IF NOT EXISTS uq_appt_active_slot ON appointments (slot_id) WHERE status = 'BOOKED';

```

C. Screenshots

Taken from the app running in Docker (docker compose up --build), logged in as sam.dev and raj.mentor

Interview Scheduler Browse Slots Log in

MENTOR-LED INTERVIEW PREP

Interview Scheduler

Book resume reviews and mock interviews with experienced engineers.

Browse open slots 9 open slots

Services

Choose the support you need

Behavioral Interview Prep STAR-method practice for behavioral questions. 45 min \$35	Mock Coding Interview LeetCode-style problem with live feedback. 60 min \$50	Mock System Design Design a large-scale system, then debrief. 60 min \$60	Resume Review Line-by-line feedback on your SWE resume. 30 min \$25
--	--	---	---

Mentors

Learn from experienced professionals

Alice Nguyen Senior Backend Engineer, 8 yrs See open slots	Maria Gonzalez Engineering Manager & Tech Recruiter See open slots	Raj Patel Staff Engineer, ex-FAANG See open slots
---	---	--

Figure 1. Home

Interview Scheduler Browse Slots Log in

Log in

Username

Password

Log in

Demo accounts (password `password123`): customers `sam.dev`, `jordan.dev`; providers `alice.mentor`, `raj.mentor`, `maria.mentor`.

Figure 2. Login

Interview Scheduler
Browse Slots
My Appointments
Sam Lee (customer)
Log out

FIND A TIME

Available Slots

Filter available sessions by service, mentor, or date.

Service
Any service

Mentor
Any mentor

Date
mm/dd/yyyy

Search

Clear

WHEN	SERVICE	MENTOR	PRICE	
Fri, 10/9, 9:30 AM	Resume Review	Maria Gonzalez	\$25	Book
Fri, 10/9, 10:00 AM	Mock Coding Interview	Alice Nguyen	\$50	Book
Fri, 10/9, 1:00 PM	Mock Coding Interview	Alice Nguyen	\$50	Book
Fri, 10/9, 3:00 PM	Mock System Design	Raj Patel	\$60	Book
Sat, 10/10, 9:00 AM	Resume Review	Alice Nguyen	\$25	Book

Prev
Page 1 of 2 (9 open)
Next

Figure 3. Available slots (filter + pagination)

Interview Scheduler
Browse Slots
My Appointments
Sam Lee (customer)
Log out

FINAL STEP

Book Appointment

Review your session details and add a note for your mentor.

APPOINTMENT SUMMARY

SERVICE	Mock System Design
MENTOR	Raj Patel
TIME	Fri, 10/9, 3:00 PM-4:00 PM
PRICE	\$60

Notes

Help your mentor prepare for your session.

Senior backend loop next month; focus on scaling a booking system

Optional, up to 500 characters.

Confirm Booking
Cancel

Figure 4. Book appointment (form)

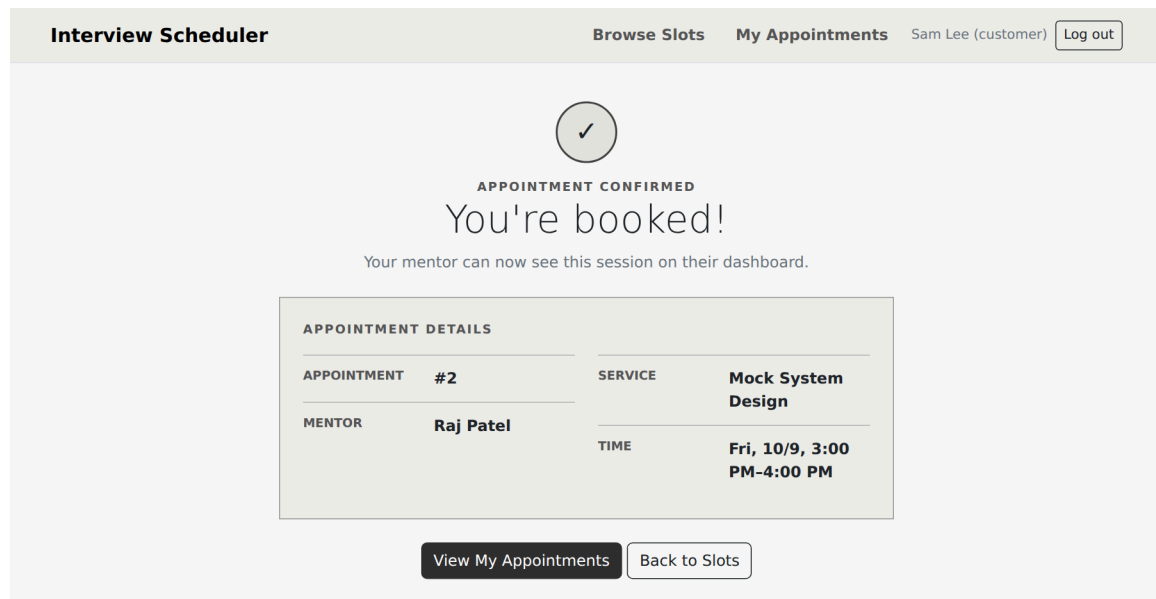


Figure 5. Confirmation

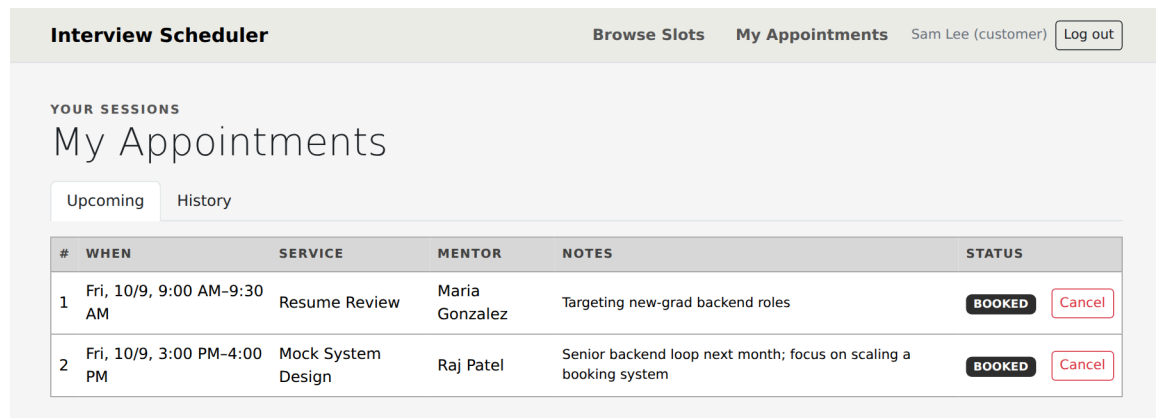


Figure 6. My appointments (upcoming; History tab next to it)

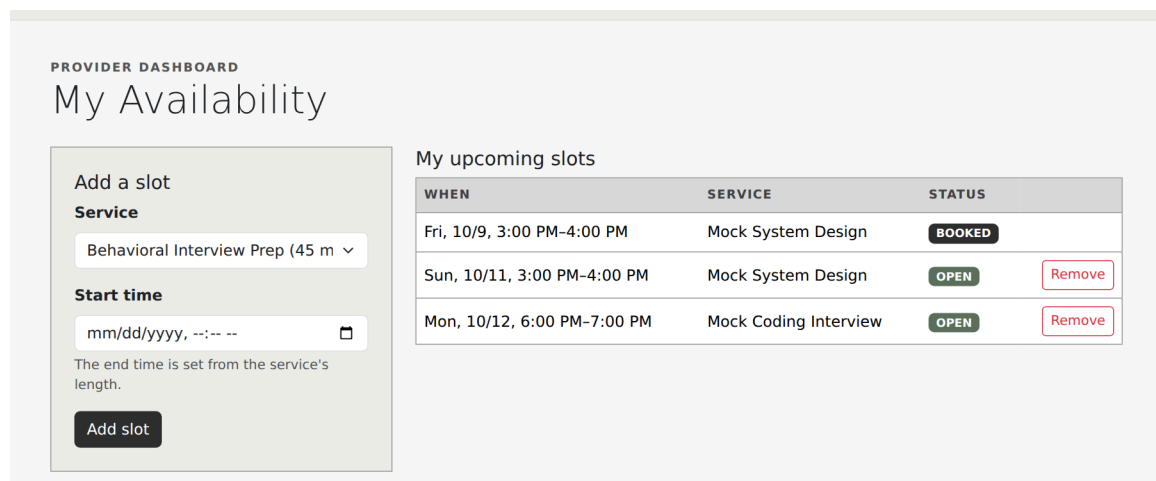


Figure 7. Provider: my slots (create / remove)

Appointments booked with me					
Upcoming		History			
#	WHEN	SERVICE	CUSTOMER	NOTES	STATUS
2	Fri, 10/9, 3:00 PM-4:00 PM	Mock System Design	Sam Lee	Senior backend loop next month; focus on scaling a booking system	BOOKED

Figure 8. Provider: booked appointments

D. Diagrams

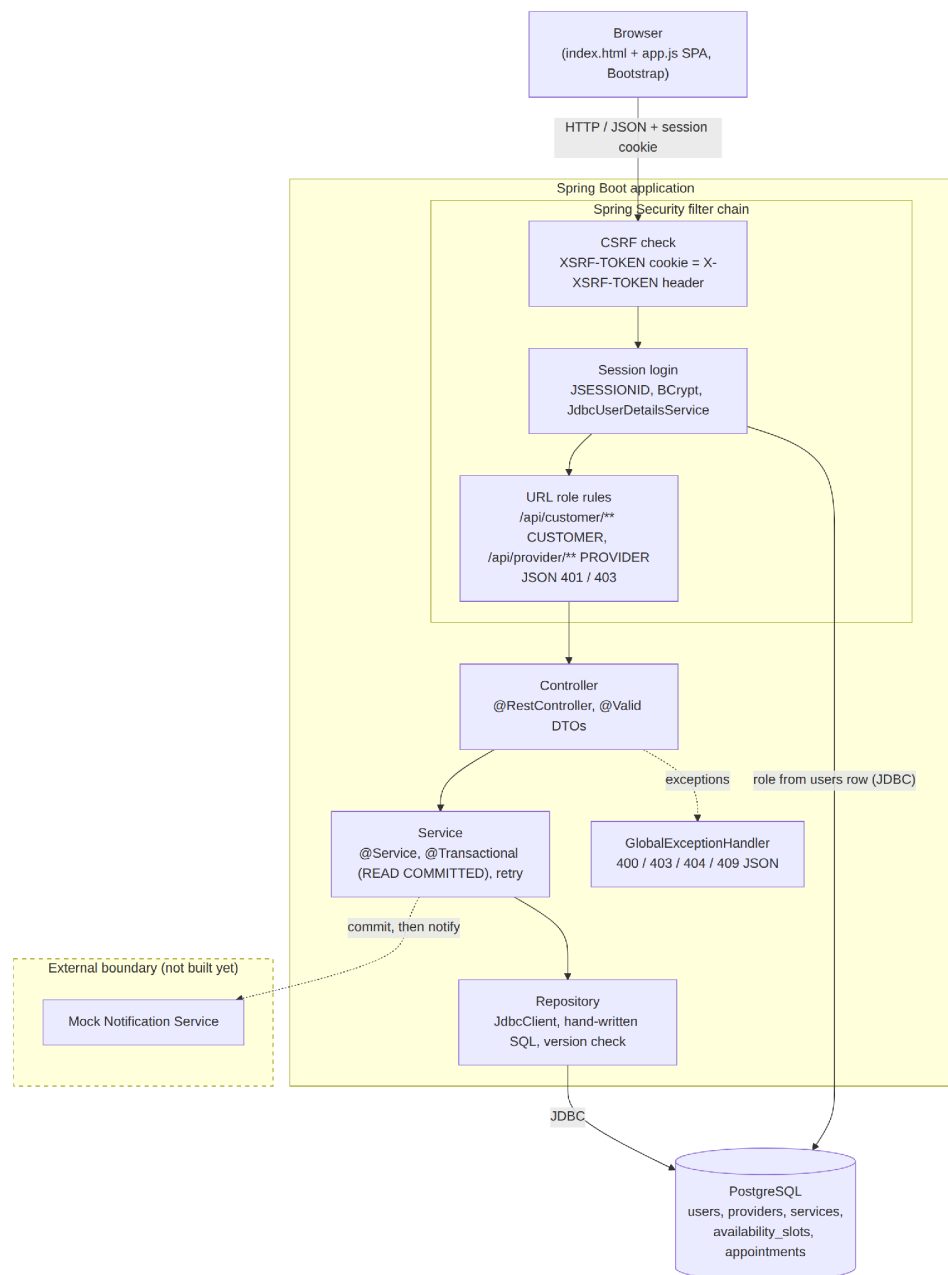


Figure 10. Block diagram with the Spring Security layer (docs/block-diagram.png)

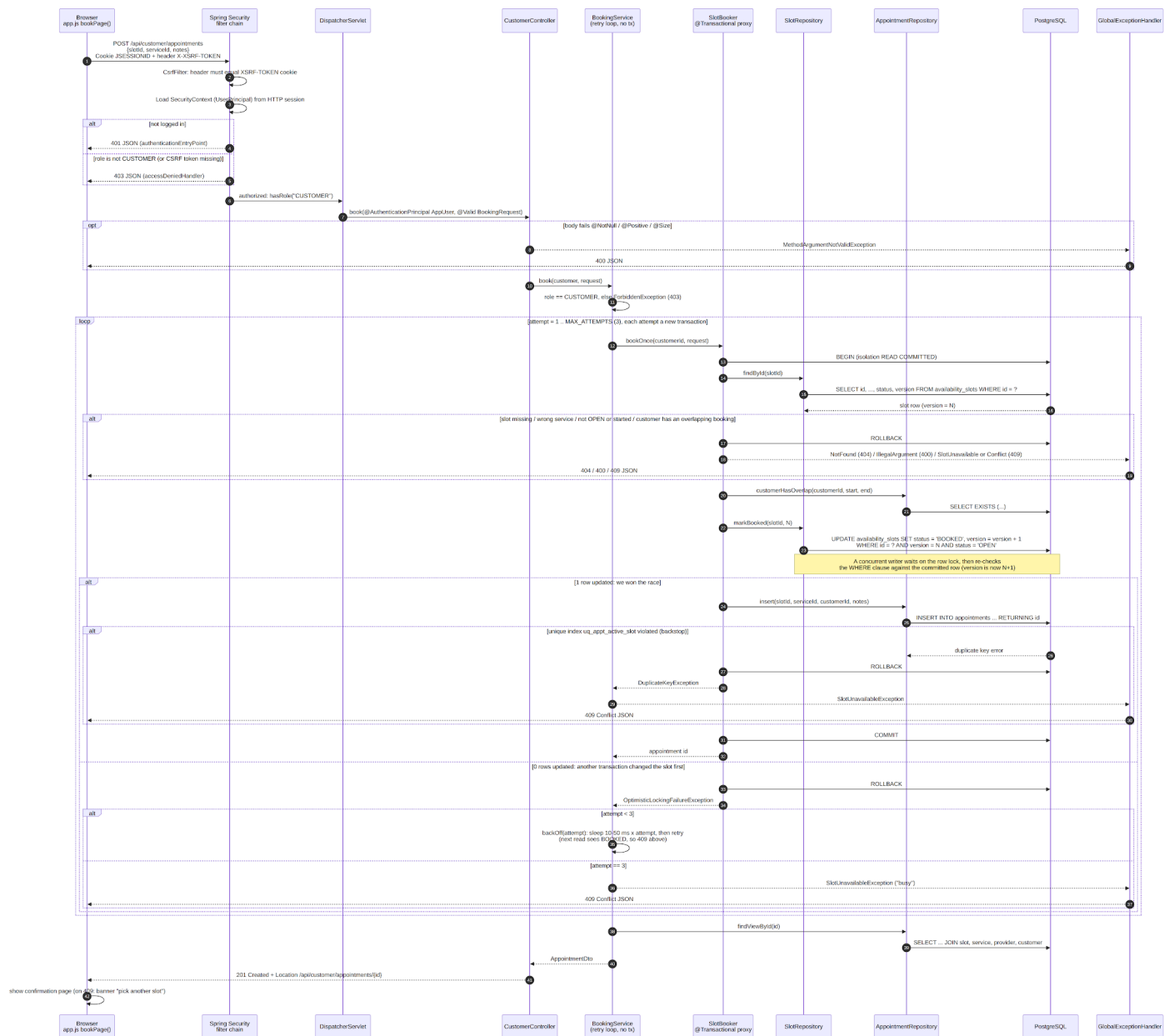


Figure 11. POST /api/customer/appointments: retry loop, transaction, 0-rows and 409 paths (docs/booking-sequence.png)